# Pinocchio and Groth16 notes

arnaucube

2026

**Abstract**

Notes taken while studying the schemes Pinocchio ([1]) and Groth16 ([2]).

Usually while reading books and papers I take handwritten notes in a notebook, this document contains some of them re-written to $LaTeX$.

The proofs may slightly differ from the ones from the books or papers, since I try to extend them for a deeper understanding.

## Contents

# 1 Introduction

# 2 Notation

Let $g \in \mathbb{G}$ be the generator of the group $\mathbb{G}$, ie. $\mathbb{G} =< g >$, then denote with $[a]$ by $g^a \in \mathbb{G}$.

# 3 R1CS

Rank-One Constraint System (R1CS) is the representation of the constraints of the circuit by 3 linear combinations, where there is one constraint per operation.

$$\begin{aligned}
(a_{11}w_1 + a_{12}w_2 + ... + a_{1n}w_n) \cdot (b_{11}w_1 + b_{12}w_2 + ... + b_{1n}w_n) \\
-(c_{11}w_1 + c_{12}w_2 + ... + c_{1n}w_n) = 0 \\
(a_{21}w_1 + a_{22}w_2 + ... + a_{2n}w_n) \cdot (b_{21}w_1 + b_{22}w_2 + ... + b_{2n}w_n) \\
-(c_{21}w_1 + c_{22}w_2 + ... + c_{2n}w_n) = 0 \\
(a_{31}w_1 + a_{32}w_2 + ... + a_{3n}w_n) \cdot (b_{31}w_1 + b_{32}w_2 + ... + b_{3n}w_n) \\
-(c_{31}w_1 + c_{32}w_2 + ... + c_{3n}w_n) = 0 \\
\dots \\
(a_{m1}w_1 + a_{m2}w_2 + ... + a_{mn}w_n) \cdot (b_{m1}w_1 + b_{m2}w_2 + ... + b_{mn}w_n) \\
-(c_{m1}w_1 + c_{m2}w_2 + ... + c_{mn}w_n) = 0
\end{aligned}$$

Where the R1CS linear combinations can be represented by three matrices $A$, $B$ and $C$:

$$A = \begin{pmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ a_{31} & a_{32} & ... & a_{3n} \\ ... & & & \\ a_{m1} & a_{m2} & ... & a_{mn} \end{pmatrix} B = \begin{pmatrix} b_{11} & b_{12} & ... & b_{1n} \\ b_{21} & b_{22} & ... & b_{2n} \\ b_{31} & b_{32} & ... & b_{3n} \\ ... & & & \\ b_{m1} & b_{m2} & ... & b_{mn} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} & ... & c_{1n} \\ c_{21} & c_{22} & ... & c_{2n} \\ c_{31} & c_{32} & ... & c_{3n} \\ ... & & & \\ c_{m1} & c_{m2} & ... & c_{mn} \end{pmatrix}$$

Where $(A, B, C) = Aw \circ Bw - Cw = 0$

The process that converts the *flat constraints* into *R1CS* is the following:

Lets assume that we have a constraint $5w_3 = w_1 \cdot 2w_2$ (being $w_1$, $w_2$ and $w_3$ the signals 1, 2 and 3 respectively)). We can represent this constraint in the matrices form by:

$$A = \begin{pmatrix} 1 & 0 & 0 & ... & 0 \end{pmatrix}$$
$$B = \begin{pmatrix} 0 & 2 & 0 & ... & 0 \end{pmatrix}$$
$$C = \begin{pmatrix} 0 & 0 & 5 & ... & 0 \end{pmatrix}$$

Which in the R1CS equation form would look as:

$$(1 \cdot w_1 + 0 \cdot w_2 + 0 \cdot w_3 + ... + 0 \cdot w_n) \cdot (0 \cdot w_1 + 2 \cdot w_2 + 0 \cdot w_3 + ... + 0 \cdot w_n) - (0 \cdot w_1 + 0 \cdot w_2 + 5 \cdot w_3 + ... + 0 \cdot w_n) = 0$$

If we simplify the previous liniar combination we get:

$$\begin{aligned} (1 \cdot w_1) \cdot (2 \cdot w_2) - (5 \cdot w_3) &= 0 \\ w_1 \cdot 2w_1 - 5w_3 &= 0 \\ 5w_3 &= w_1 \cdot 2w_1 \end{aligned}$$

Where $5w_3 = w_1 \cdot 2w_1$ is the constraint that we wanted to represent in the R1CS format.

The nice thing from R1CS is that allows us to standaridze the constraints in an equation fashion, where all the signals are involved and we 'activate' these ones that are used in each constraint. Since we have now the constraints in a polynomial format, we can do work with it in the polynomial way.

# 4 QAP (Quadratic Arithmetic Programs)

## 4.1 QAP

Once we have the constraints of a circuit represented in *R1CS*, we can do one more iteration and represent it in *Quadratic Arithmetic Programs (QAP)*. *QAP* is a representation of the XXX in a linear combination of the *R1CS*, represented by 3 polynomials $a(X)$, $b(X)$ and $c(X)$. The *QAP* consists of the 3 polynomials $a(X)$, $b(X)$ and $c(X)$, plus a divisor polynomial $h(X) \in \mathbb{F}[X]^{<d}$.

The main idea behind the *QAP* is that they 'contain' all the constraints of the circuit, meaning that these polynomials need to be able to generate all the constraints.

In order to 'collapse' the *R1CS* polynomials into the 3 polynomials of the *QAP*, we make use of *polynomial interpolation*, also named *Lagrange interpolation*. *Lagrange interpolation* allows for a group of points, to find the smallest degree and unique polynomial that goes through all those points. It can be calculated by computing:

$$L(X) = \sum_{j=0}^{n} y_j l_j(X)$$

where $l_j(X)$ is:

$$l_j(X) = \prod_{0 \leq m \leq k; m \neq j} \frac{X - x_m}{x_j - x_m} ... \frac{(X - x_{j-1})}{(x_j - x_{j-1})} \frac{(X - x_{j+1})}{(x_j - x_{j+1})} ... \frac{X - x_k}{x_j - x_k}$$

The intuition behind the *Lagrange interpolation* is quite simple. If we have two points (on a 2 dimensional plane) we can find a line that goes through both points (this can be done with a pen and a ruler on a paper), if we have 3 points we can find a curved line that goes through all the three points, if we do this through math trying to get the lowest degree curve we obtain a 2nd degree polynomial. This can be generalized using the *Lagrange interpolation*, allowing us for a group of $n$ points to find a $n-1$ degree polynomial that goes through all of them.

In our case, *Lagrange interpolation* is used to go from the *R1CS* polynomials into the *QAP* polynomials, which are defined by:

$$a(X) = (a_1(X)w_1 + a_2(X)w_2 + ... + a_n(X)w_n) = \sum_{i=1}^{n} a_i(X)w_i$$
$$b(X) = (b_1(X)w_1 + b_2(X)w_2 + ... + b_n(X)w_n) = \sum_{i=1}^{n} b_i(X)w_i$$
$$c(X) = (c_1(X)w_1 + c_2(X)w_2 + ... + c_n(X)w_n) = \sum_{i=1}^{n} c_i(X)w_i$$

The polynomials $a(X), b(X), c(X)$ are the QAP, such that

$$a(X) \cdot b(X) = c(X)$$

so that

$$p(X) = a(X) \cdot b(X) - c(X) = 0$$

that is,

$$\begin{aligned} p(X) =& (a_1(X)w_1 + a_2(X)w_2 + ... + a_n(X)w_n) \\ & \cdot (b_1(X)w_1 + b_2(X)w_2 + ... + b_n(X)w_n) \\ & - (c_1(X)w_1 + c_2(X)w_2 + ... + c_n(X)w_n) \\ =& (\sum_{i=1}^{n} a_i(X)w_i) \cdot (\sum_{i=1}^{n} b_i(X)w_i) - (\sum_{i=1}^{n} c_i(X)w_i) = 0 \end{aligned}$$

The main idea, is that $p(X)$ defines the constraints of our circuit, so that when evaluating $p(X)$ at the challenge, it will be equal to zero.

## 4.2 QAP Relation

Want to ensure that

$$(\sum_{i=1}^{n} a_i(X)w_i) \cdot (\sum_{i=1}^{n} b_i(X)w_i) - (\sum_{i=1}^{n} c_i(X)w_i) = 0$$

for a given witness $(w_1, \ldots, w_n) \in \mathbb{F}^n$, which we divide between public inputs and private inputs:

$$\begin{aligned} &\text{public inputs: } io = \phi = (w_1, \ldots, w_l) \in \mathbb{F}^l, \\ &\text{private inputs: } s = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l-1}, \end{aligned}$$

We denote the relation

$$R_{QAP} = \begin{cases} w = (1, \phi, s) \in \mathbb{F}^n \text{ such that:} \\ \phi = (w_1, \ldots, w_l) \in \mathbb{F}^l, \ s = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l-1}, \\ (\sum_{i=0}^{n} a_i(X) \cdot w_i) \cdot (\sum_{i=0}^{n} b_i(X) \cdot w_i) = \sum_{i=0}^{n} c_i(X) \cdot w_i \mod Z(X) \end{cases}$$

where $Z(X)$ is the "zero polynomial" or "vanishing polynomial", such that

$$Z(X) = \prod_{i=0}^{d} (X - x_i) \in \mathbb{F}[X]^{<d}$$

Set

$$a(X) = \sum_{i=1}^{n} a_i(X) w_i, \quad b(X) = \sum_{i=1}^{n} b_i(X) w_i), \quad c(X) = \sum_{i=1}^{n} c_i(X) w_i$$

so that

$$a(X) \cdot b(X) \equiv c(X) \bmod Z(X)$$

which means that we can prove that

$$\exists\ h(X) \in \mathbb{F}[X]^{<d} \text{ s.th. } a(X) \cdot b(X) - c(X) = h(X) \cdot Z(X)$$

# 5 Preliminar protocols

## 5.1 Sigma protocol

Let $q$ be a prime, $q$ a prime divisor in $p-1$, and $g$ and element of order $q$ in $\mathbb{Z}_p^a$. Then we have $G = \langle g \rangle$.
We assume that computationally for a given $A$ it's hard to find $a \in \mathbb{F}$ such that $A = g^a$.
Alice wants to prove that knows the *witness* $w \in \mathbb{F}$, such that the *statement* $X = g^w$, without revealing $w$.

1. Alice generates a random $a \in^R \mathbb{F}$, and computes $A = g^a$. And sends $A$ to Bob.

2. Bob generates a challenge $c \in^R \mathbb{F}$, and sends it to Alice.

3. Alice computes $z = a + c \cdot w$, and sends it to Bob.

4. Bob verifies it by checking that $g^z == X^c \cdot A$.

   We can unfold Bob's verification and see that:

$$g^z == X^c \cdot A$$

$$g^{a+cw} == g^{wc} g^a$$

$$g^{a+cw} == g^{wc+a}$$

   Properties:

   i. *correctness/completness*: if Alice know the witness for the statement, then they can create a valid proof.

   ii. *soundness*: if someone does not have knowledge of the witness, can not form a valid proof (verifier will always reject).

   iii. *zero knowledge*: nobody gains knowledge of anything new with the proof. prior knowledge + proof = prior knowledge

**Non interactive protocol:** With the *Fiat-Shamir Heuristic*, we model a hash function as a random oracle, thus we can replace Bob's role by a hash function in order to obtain the challenge $c \in \mathbb{F}$.
So, we replace the step 2 from the described protocol by $c = H(X||A)$ (where $H$ is a known hash function).

**What could go wrong (Simulator).** If the verifier (Bob) sends $c \in \mathbb{F}$, prior to the prover committed to $A$, the prover could create a proof about a public key which they don't know $w$.

1. Bob sends $c \xleftarrow{r} \mathbb{F}$ to Alice

2. Alice generates $z \xleftarrow{r} \mathbb{F}$

3. Alice then computes $A = g^z X^{-c}$, and sends $z, A$ to Bob

4. Bob would check that $g^z == X^c A$ and it would pass the verification, as $g^z == X^c \cdot A \Rightarrow g^z == X^c \cdot g^z X^{-c} \Rightarrow g^z == g^z$.

As we've seen, it's really important the order of the steps, so Alice must commit to $A$ before knowing $c$.
This 'fake' proof generation is often called the *simulator* and used for further constructions.

## 5.2 Proof of Exponent

This is a protocol used for when the prover want to convince the verifier about $p(X)|t(X)$. Assume that we have $g^{\tau^i}$.

The prover can compute

$$g^{p(\tau)} = \sum (g^{\tau^i})^{p_i}$$
$$g^{h(\tau)} = \sum (g^{\tau^i})^{h_i}$$
$$g^{t(\tau)} = \sum (g^{\tau^i})^{t_i}$$

where $p_i, h_i, t_i$ are the coefficients of $p(X), h(X), t(X)$ respectively.

So that the verifier can check

$$g^{p(\tau)} = (g^{h(\tau)})^{t(\tau)}$$

through a pairing check as

$$e([p(\tau)], [1]) = e([h(\tau)], [t(\tau)])$$

Notice that this is not enough, since the prover could use a $h(X)$ such that $h(\tau) = r$, and then produce a valid proof where $g^{r \cdot t(\tau)} = (g^r)^{t(\tau)}$.

The solution is to add the $\alpha$-shifted evaluation:

The verifier would then check

$$g^{p(\alpha \cdot \tau)} = (g^{p(\tau)})^{\alpha}$$
$$g^{h(\alpha \cdot \tau)} = (g^{h(\tau)})^{\alpha}$$
$$g^{t(\alpha \cdot \tau)} = (g^{t(\tau)})^{\alpha}$$

through the pairing checks as

$$e([p(\alpha \cdot \tau)], [1]) = e([p(\tau)], [\alpha])$$
$$e([h(\alpha \cdot \tau)], [1]) = e([h(\tau)], [\alpha])$$
$$e([t(\alpha \cdot \tau)], [1]) = e([t(\tau)], [\alpha])$$

## 5.3 Simplified QAP check protocol

Protocol to prove $a(X)b(X) - c(X) = 0$ in order to provide intuition on what we're trying to do.

**Setup.** The protocol requires a secret random value, which it is encrypted over the elliptic curve group. That is, for given $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ generators of curves $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively, set $[\tau]_1 = g_1^{\tau}$ and $[\tau]_2 = g_2^{\tau}$.

$$\{[\tau^i]_1\}_0^n = ([\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-1}]_1)$$
$$\{[\tau^i]_2\}_0^n = ([\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{n-1}]_2)$$

**Prover.**

i. commit to $a(X), b(X), c(X) \in \mathbb{F}[X]$

$$cm(a) = [a(\tau)]_1 = \sum_{i=0}^{\delta a} a_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

notice that it is equal to $\sum a_i \cdot g_1^{\tau^i}$ without knowing $\tau$.
Same is done for $b(X)$ and $c(X)$:

$$cm(b) = [b(\tau)]_2 = \sum_{i=0}^{\delta b} b_i \cdot [\tau^i]_2 \in \mathbb{G}_2$$

$$cm(c) = [c(\tau)]_1 = \sum_{i=0}^{\delta c} c_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

For a challenge $z \in^R \mathbb{F}$, set by the verifier, open the commitments:

$$q_a(X) = \frac{a(X) - a(z)}{X - z}$$

Since $a(X) - \underbrace{a(z)}_{=y}$ should be zero at $X - z$, so $a(X) - a(z)$ has a root at $z$, hence it is divisible by $(X - z)$, therefore,

$$\exists\ q_a(X) \ \text{ such that } a(X) - a(z) = (X - z) \cdot q_a(X)$$

therefore

$$q_a(X) = \frac{a(X) - a(z)}{X - z}$$

Same is done for $b(X)$ and $c(X)$,

$$q_b(X) = \frac{b(X) - b(z)}{X - z}$$

$$q_c(X) = \frac{c(X) - c(z)}{X - z}$$

And then, evaluate them at the encrypted $\tau$:

$$[q_a(\tau)]_1 = \sum_{i=0}^{\delta q_a} q_{a_i} \cdot [\tau^i]_1 \in \mathbb{G}_1$$

similarly for $[q_b(\tau)]_2 \in \mathbb{G}_2$ and $[q_c(\tau)]_1 \in \mathbb{G}_1$.

ii. for the relation that we want to prove, $a(X) \cdot b(X) - c(X) = 0$,

$$\exists\ q(X) \in \mathbb{F}[X] \ \text{ s.th. } \ a(X)b(X) - c(X) = q(X) \cdot z(X)$$

So the prover computes

$$q(X) = \frac{a(X) \cdot b(X) - c(X)}{z(X)}$$

and it's encrypted evaluation

$$[q(\tau)]_1 = \sum_{i=0}^{\delta q} q_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

**Verifier.**

i. verify the commitments:
The prover has sent the commitments $[a(\tau)]_1$, $[b(\tau)]_2$, $[c(\tau)]_1$, and their respective evaluations at the challenge $z \in \mathbb{F}$: $a(z), b(z), c(z) \in \mathbb{F}$; together with their respective opening proofs: $[q_a(\tau)]_1, [q_b(\tau)]_2, [q_c(\tau)]_1$

For each commitment, the verifier checks:

$$e([q_a(\tau)]_1, [\tau]_2 - [z]_2) \stackrel{?}{=} e([a(\tau)]_1 - [a(z)]_1, [1]_2)$$

which is a way of checking the existence of $q_a(X)$ such that $q_a(X) \cdot (X - z) = a(X) - a(z)$.

If the checks passes, it means that $q_a \in \mathbb{F}[X]$ exists, and thus $a(X) - a(z)$ is divisible ("vanishes") at $(X - z)$, hence $a(X) - a(z)$ has a root at $z$.

Same is done for $b(X)$ and $c(X)$ commitments:

$$e([\tau]_1 - [z]_2, [q_b(\tau)]_1) \stackrel{?}{=} e([1]_1, [b(\tau)]_1 - [b(z)]_1)$$

$$e([q_c(\tau)]_1, [\tau]_2 - [z]_2) \stackrel{?}{=} e([c(\tau)]_1 - [c(z)]_1, [1]_2)$$

ii. check the $a(X) \cdot b(X) - c(X) = 0$ relation:

$$e([a(\tau)]_1, [b(\tau)]_2) \stackrel{?}{=} e([q(\tau)]_1, [z(\tau)]_2) \cdot e([c(s)]_1, [1]_2)$$

which ensures that $\exists\ q(X) \in \mathbb{F}[X]$ s.th. $a(X)b(X) - c(X) = q(X)z(X)$.

# 6 Pinocchio

The next couple of subsections provide first a description of the conceptual protocol, and then the optimized version of it, which reduces prover costs and key sizes.

## 6.1 Conceptual version

*(this is what the paper names "Protocol 1")*

Recall, from 4.2, public inputs: $\phi = (w_1, \ldots, w_l) \in \mathbb{F}^l$, and private inputs: $s = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l-1}$.

Let $[m] = \{l+1, \ldots, n\}$.

**SRS** Let $\tau, \alpha, \beta_a, \beta_b, \beta_c, \gamma \in^R \mathbb{F}$ be the *secret* challenges for the protocol.

The *proving key* will be

$$\begin{aligned}
&\{[a_i(\tau)]\}_{i\in[m]},\ \{[b_i(\tau)]\}_{i\in[n]},\ \{[c_i(\tau)]\}_{i\in[n]}\\
&\{[\alpha \cdot a_i(\tau)]\}_{i\in[m]},\ \{[\alpha \cdot b_i(\tau)]\}_{i\in[n]},\ \{[\alpha \cdot c_i(\tau)]\}_{i\in[n]}\\
&\{[\beta_a \cdot a_i(\tau)]\}_{i\in[m]},\ \{[\beta_b \cdot b_i(\tau)]\}_{i\in[n]},\ \{[\beta_c \cdot c_i(\tau)]\}_{i\in[n]}\\
&\{[\tau^i]\}_{i\in[d]},\ \{[\alpha \cdot \tau^i]\}_{i\in[d]}
\end{aligned}$$

The *verifying key* will be

$$\begin{aligned}
&([1], [\alpha], [\gamma], [\beta_a \cdot \gamma], [\beta_b \cdot \gamma], [\beta_c \cdot \gamma], [Z(\tau)],\\
&\{[a_i(\tau)]\}_{i\in l}, [a_0(\tau)], [b_0(\tau)], [c_0(\tau)])
\end{aligned}$$

**Prover** Solve for $h(X)$ such that $p(X) = h(X) \cdot t(X)$, then

Let

$$a_m(X) = \sum_{i\in[m]} w_i \cdot a_i(X)$$

Denote $a(X), b(X), c(X)$ as

$$a(X) = \sum_{i\in[n]} w_i \cdot a_i(X), \quad b(X) = \sum_{i\in[n]} w_i \cdot b_i(X)$$
$$c(X) = \sum_{i\in[n]} w_i \cdot c_i(X)$$

Then the proof $\pi$ is

$$\begin{aligned}
\pi = (&[a_m(\tau)],\ [b(\tau)],\ [c(\tau)],\ [h(\tau)],\\
&[\alpha \cdot a_m(\tau)],\ [\alpha \cdot b(\tau)],\ [\alpha \cdot c(\tau)],\ [\alpha \cdot h(\tau)],\\
&\quad \textit{(ie. same as first line but } \alpha\textit{-shifted)}\\
&[\beta_a \cdot a(\tau) + \beta_b \cdot b(\tau) + \beta_c \cdot c(\tau)])
\end{aligned}$$

Since $a(X), b(X), c(X)$ are linear equations, we can compute them in the exponent of $g$, eg.

$$g^{a(\tau)} = \prod_{i \in n} (g^{a_i(\tau)})^{w_i}$$

which in our notation is

$$[a(\tau)] = \sum_{i \in n} [a_i(\tau) \cdot w_i]$$

**Verifier** First check that $\alpha, \beta$ related terms are correct:

$\alpha$-terms:

$$\begin{aligned} e([a_m(\tau)], [\alpha]) &= e([\alpha \cdot a_m(\tau)], [1]) \\ e([b(\tau)], [\alpha]) &= e([\alpha \cdot b(\tau)], [1]) \\ e([c(\tau)], [\alpha]) &= e([\alpha \cdot c(\tau)], [1]) \\ e([h(\tau)], [\alpha]) &= e([\alpha \cdot h(\tau)], [1]) \end{aligned}$$

which we can group as:

$$\prod_{p \in \{a_m, b, c, h\}} e([p(\tau)], [\alpha]) = \prod_{p \in \{a_m, b, c, h\}} e([\alpha \cdot p(\tau)], [1])$$

$\beta$-terms:

$$\begin{aligned} e([\beta_a \cdot a(\tau) + \beta_b \cdot b(\tau) + \beta_c \cdot c(\tau)], [1]) = \\ e([a_m(\tau)], [\beta_a]) \cdot e([b(\tau)], [\beta_b]) \cdot e([c(\tau)], [\beta_c]) \end{aligned}$$

This is, 8 pairings for $\alpha$ terms, and 3 pairings for the $\beta$ terms.

Then, computes the term representing the public inputs:

$$g^{a_{io}} = \prod_{i \in l} (g^{a_i(\tau)})^{w_i}$$

ie.

$$[a_{io}] = \sum_{i \in l} [a_i(\tau) \cdot w_i]$$

in order to do the final check, verifying the divisibility, and therefore the QAP relation:

$$e([a_0(\tau) + a_{io} + a(\tau)], [b_0(\tau) + b(\tau)]) = e([c_0(\tau) + c(\tau)], [1]) \cdot e([h(\tau)], [Z(\tau)])$$

## 6.2 Actual Pinocchio protocol

Recall, from 4.2, public inputs: $\phi = (w_1, \ldots, w_l) \in \mathbb{F}^l$, and private inputs: $s = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l-1}$.

Let $[m] = \{l+1, \ldots, n\}$.

**SRS** Let $r_a, r_b, \tau, \alpha_a, \alpha_b, \alpha_c, \beta, \gamma \in^R \mathbb{F}$ be the *secret* challenges for the protocol.
Also with $r_c = r_a \cdot r_b,\ g_a = g^{r_a},\ g_b = g^{r_b},\ g_c = g^{r_c}$.
The *proving key* will be

$$\{g_a^{a_i(\tau)}\}_{i\in[m]},\ \{[g_b^{b_i(\tau)}\}_{i\in[m]},\ \{[g_c^{c_i(\tau)}\}_{i\in[m]}$$
$$\{g_a^{\alpha_a \cdot a_i(\tau)}\}_{i\in[m]},\ \{[g_b^{\alpha_b \cdot b_i(\tau)}\}_{i\in[m]},\ \{[g_c^{\alpha_c \cdot c_i(\tau)}\}_{i\in[m]}$$
$$\{g^{\tau^i}\}_{i\in[d]},\ \{g_a^{\beta a_i(\tau)} \cdot g_b^{\beta b_i(\tau)} \cdot g_c^{\beta c_i(\tau)}\}_{i\in[m]}$$

ie, in our notation,

$$\{[r_a \cdot a_i(\tau)]\}_{i\in[m]},\ \{[r_b \cdot b_i(\tau)]\}_{i\in[m]},\ \{[r_c \cdot c_i(\tau)]\}_{i\in[m]}$$
$$\{[r_a \cdot \alpha_a \cdot a_i(\tau)]\}_{i\in[m]},\ \{[r_b \cdot \alpha_b \cdot b_i(\tau)]\}_{i\in[m]},\ \{[r_c \cdot \alpha_c \cdot c_i(\tau)]\}_{i\in[m]}$$
$$\{[\tau^i]\}_{i\in[d]},\ \{[r_a \cdot \beta \cdot a_i(\tau) + r_b \cdot \beta \cdot b_i(\tau) + r_c \cdot \beta \cdot c_i(\tau)\}_{i\in[m]}$$

The *verifying key* will be

$$([1], [\alpha_a], [\alpha_b], [\alpha_c], [\gamma], [\beta\gamma], [r_c \cdot t(\tau)],$$
$$\{[r_a \cdot a_i(\tau)],\ [r_b \cdot b_i(\tau)],\ [r_c \cdot c_i(\tau)]\}_{i\in\{0\}\cup[l]})$$

**Prover** Solve for $h(X)$ such that $p(X) = h(X) \cdot t(X)$, then
The proof $\pi$ is

$$\pi = ([r_a \cdot a_m(\tau)],\ [r_b \cdot b_m(\tau)],\ [r_c \cdot c_m(\tau)],\ [h(\tau)],$$
$$[r_a \cdot \alpha \cdot a_m(\tau)],\ [r_b \cdot \alpha \cdot b(\tau)],\ [r_c \cdot \alpha \cdot c(\tau)],$$
$$[r_a \cdot \beta \cdot a_m(\tau) + r_b \cdot \beta \cdot b_m(\tau) + r_c\beta \cdot c_m(\tau)]$$

where

$$a_m(X) = \sum_{i\in[m]} w_i \cdot a_i(X)$$

and similarly for $b_m(X),\ c_m(X)$.
Note that the last element of $\pi$ is $g^z$ (*ie.* $[z]$) later at the verifier.

**Verifier** The verifier receives the proof

$$\pi = (g^{a_m}, g^{b_m}, g^{c_m}, g^h, g^{a'_m}, g^{b'_m}, g^{c'_m}, g^z)$$

ie.

$$\pi = ([a_m], [b_m], [c_m], [h], [a'_m], [b'_m], [c'_m], [z])$$

First, it does the divisibility check:

compute

$$g_a^{a_{io}(\tau)} = \prod_{i \in l} (g_a^{a_{io}(\tau)})^{w_i}$$

ie.

$$[r_a \cdot a_{io}(\tau)] = [\sum_{i \in l} (r_a \cdot a_{io}(\tau) \cdot w_i)]$$

similarly for $g_b^{b_{io}(\tau)},\ g_c^{c_{io}(\tau)}$.

And then it checks

$$e(g_a^{a_0(\tau)} \cdot g_a^{a_{io}(\tau)} \cdot g_a^{a_m},\ g_b^{b_0(\tau)} \cdot g_b^{b_{io}(\tau)} \cdot g_b^{b_m}) = e(g_c^{c_0(\tau)} \cdot g_c^{c_{io}(\tau)} \cdot g_c^{c_m}), g^1)$$

ie.

$$e([r_a \cdot (a_0(\tau) \cdot a_{io}(\tau) \cdot a_m)]_1,\ [r_b \cdot (b_0(\tau) \cdot b_{io}(\tau) \cdot b_m)]_2) = e([r_c \cdot (c_0(\tau) \cdot c_{io}(\tau) \cdot c_m)]_1, [1]_2)$$

Next, it checks that the linear combinations computed over the QAP are in their appropriate spans:

$$e([r_a \cdot a'_m]_1, [1]_2) = e([r_a \cdot a_m]_1, [\alpha_a]_2)$$
$$e([r_b \cdot b'_m]_1, [1]_2) = e([r_b \cdot b_m]_1, [\alpha_b]_2)$$
$$e([r_c \cdot c'_m]_1, [1]_2) = e([r_c \cdot c_m]_1, [\alpha_c]_2)$$

And finally, it checks that the same coefficients were used in each of the linear combinations over the QAP

$$e(g^z, g^\gamma) = e(g_a^{a_m} \cdot g_b^{b_m} \cdot g_c^{c_m}, g^{\beta\gamma})$$

ie.

$$e([z]_1, [\gamma]_2 = e([r_a \cdot a_m]_1 \cdot [r_b \cdot b_m]_1 \cdot [r_c \cdot c_m]_1, [\beta \cdot \gamma]_2)$$

recall that $[z]_1$ comes from the proof.

# 7 Groth16

## 7.1 Conceptual version

As before, we want to prove the QAP relation $R_{QAP}$ (4.2).

We have, public inputs: $\phi = (w_1, \ldots, w_l) \in \mathbb{F}^l$, and private inputs: $s = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l-1}$.

The prover will solve for $h(X)$ such that $p(X) = h(X) \cdot t(X)$, then we have

$$(\sum_{i=1}^{n} a_i(X) w_i) \cdot (\sum_{i=1}^{n} b_i(X) w_i) = (\sum_{i=1}^{n} c_i(X) w_i) + h(X) \cdot t(X)$$

where $t(X)$ is the zero polynomial.

**SRS** Let $\alpha, \beta, \gamma, \delta, x \in^R \mathbb{F}$ be the *secret* challenges for the protocol, we denote them by $\tau = (\alpha, \beta, \gamma, \delta, x)$.

Then

$$\sigma = \left( \alpha, \beta, \gamma, , \{x^i\}_{i=0}^{n-1}, \left\{ \frac{\beta a_i(x) + \alpha b_i(x) + c_i(x)}{\gamma} \right\}_{i=0}^{l}, \right.$$
$$\left. \left\{ \frac{\beta a_i(x) + \alpha b_i(x) + c_i(x)}{\delta} \right\}_{i=0}^{l}, \left\{ \frac{x^i t(x)}{\delta} \right\}_{i=0}^{n-2} \right)$$

**Prover**

# 8 Powers of tau (trusted setup)

## 8.1 Main idea

## 8.2 Participant contribution

In the context of SRS (Structured Reference String), the powers of tau consist of an array containing the scalar multiplication of the generator point by each power of tau, eg. $\tau^0 \cdot G,\ \tau^1 \cdot G,\ \tau^2 \cdot G,\ \tau^3 \cdot G \ldots$

For our case, we'll generate $n$ powers of tau on $\mathbb{G}_1$, and $m$ on $\mathbb{G}_2$. We will denote the combination of both $\mathbb{G}_1$ and $\mathbb{G}_2$ powers of tau by SRS.

Each participant will obtain the previous-participant generated SRS, which is formed by

$$\{[\tau^0]_1, [\tau^1]_1, [\tau^2]_2, \ldots, [\tau^{n-1}]_1\},$$
$$\{[\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{m-1}]_2\}$$

for simplicity, we will represent it as

$$\{\ [\ \tau^i]_1\ \}_{i=0}^{n-1},\ \{\ [\ \tau^i]_2\ \}_{j=0}^{m-1}$$

Each participant generates their secret random tau $\tau_p$, which to avoid confusion we will denote as $p \in \mathbb{F}_r$. From it, the participant can compute $[p]_2 \in \mathbb{G}_2$.

Then the participant proceeds to update the reference string (SRS), by multiplying each element by $p^i$:

$$\{\ p^i \cdot [\ \tau^i]_1\ \}_{i=0}^{n-1},\ \{\ p^i \cdot [\ \tau^i]_2\ \}_{j=0}^{m-1}$$

which, by the properties of scalar multiplication of a point, is equivalent to

$$\{\ [\ (p \cdot \tau)^i]_1\ \}_{i=0}^{n-1},\ \{\ [\ (p \cdot \tau)^i]_2\ \}_{j=0}^{m-1}$$

and we denote $p \cdot \tau$ as $\tau'$, so we can write the previous expression as

$$\{ [ \tau'^i]_1 \}_{i=0}^{n-1}, \{ [ \tau'^i]_2 \}_{j=0}^{m-1}$$

which is

$$\{[(\tau \cdot p)^0]_1, [(\tau \cdot p)^1]_1, [(\tau \cdot p)^2]_2, \ldots, [(\tau \cdot p)^{n-1}]_1\},$$
$$\{[(\tau \cdot p)^0]_2, [(\tau \cdot p)^1]_2, [(\tau \cdot p)^2]_2, \ldots, [(\tau \cdot p)^{m-1}]_2\}$$

Notice that the contributor does not know the previous $\tau$, but can obtain $(\tau \cdot p)^i$.

## 8.3 Proof of correct computation

The participant will compute the proof, which consists of

$$\pi = ([\tau']_1, \ [p]_2)$$

which is equivalent to $\pi = (\tau' \cdot G, \ p \cdot H) = ((p \cdot \tau) \cdot G, \ p \cdot H)$

In the next section we will see how this is used to verify the contribution correct computation.

## 8.4 Contribution verification

This is the interesting part. We want to check that the new SRS is well computed from the previous SRS together with the new tau. We want to avoid accepting contributions that do not follow the powers of tau structure, or that contain empty elements or that do not derive from the previous SRS.

Following the powers of tau structure means that

$$\tau'^{i+1} = \tau' \cdot \tau'^i \quad \forall \ i \in [1, n-1]$$

And the relation between the new SRS and the previous SRS is

$$\tau' = \tau \cdot p$$

where $\tau$ is the secret element from the previous SRS and $p$ is the secret one from the new SRS, resulting in the combined $\tau'$.

Now, we're interested into how we can check these relations in the context of the SRS elements. A new contribution can be verified as follows:

1. Check that elements of the new SRS are non-empty, non-zero and in the correct prime order subgroups.

2. Get the proof ($\pi = ([\tau']_1, [p]_2)$), and confirm that the 1st element is equal to the element in position 1 $[\tau'^1]_1$ of the new SRS (which corresponds to $\tau'$ powered to 1).

3. Check that $\tau'$ (the new SRS) is correctly related to $\tau$ (the previous SRS).

   To check this, we rely on the pairing properties and we use the proof value generated by the contributor, which consists of $[p]_2$

   $$e([\tau]_1, [p]_2) \stackrel{?}{=} e([\tau']_1, [1]_2)$$

   We can see how this holds, as $\tau' = p \cdot \tau$, and by the bilinear property of the pairing (see [3]) we have $e(a \cdot G, b \cdot H) = e((a \cdot b) \cdot G, H)$, which in our notation is represented as

   $$e([a]_1, [b]_2) = e([a \cdot b]_1, [1]_2)$$

   by applying this to our case we can see that:

   $$e([\tau]_1, [p]_2) = e([\tau \cdot p]_1, [1]_2) = e([\tau']_1, [1]_2)$$

4. Check if the new SRS follows the powers of tau structure:

   $$e([\tau'^i]_1, [\tau']_2) \stackrel{?}{=} e([\tau'^{i+1}]_1, [1]_2) \quad \forall\ i \in [1, n-1]$$

   $$e([\tau']_1, [\tau'^j]_2) \stackrel{?}{=} e([1]_1, [\tau'^{j+1}]_2) \quad \forall\ i \in [1, m-1]$$

   Which, again, by the bilinarity property, we can see that

   $$e([\tau'^i]_1, [\tau']_2) = e([\tau'^{i+1}]_1, [1]_2)$$

   is enforcing that $\tau'^i \cdot \tau'^1 = \tau'^{i+1}$.

   And the same for

   $$e([\tau']_1, [\tau'^j]_2) = e([1]_1, [\tau'^{j+1}]_2)$$

   which checks that $\tau'^1 \cdot \tau'^j = \tau'^{j+1}$.

   With this check, we're ensuring that each power of tau of the SRS is consistent with the previous one.

# References


[1] Bryan Parno, Craig Gentry, Jon Howell, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. Cryptology ePrint Archive, Paper 2013/279, 2013.

[2] Jens Groth. On the size of pairing-based non-interactive arguments. Cryptology ePrint Archive, Paper 2016/260, 2016.

[3] `https://github.com/arnaucube/math/blob/master/weil-pairing.pdf`.